SYSTEM DESIGN

Idempotency Visual Guide

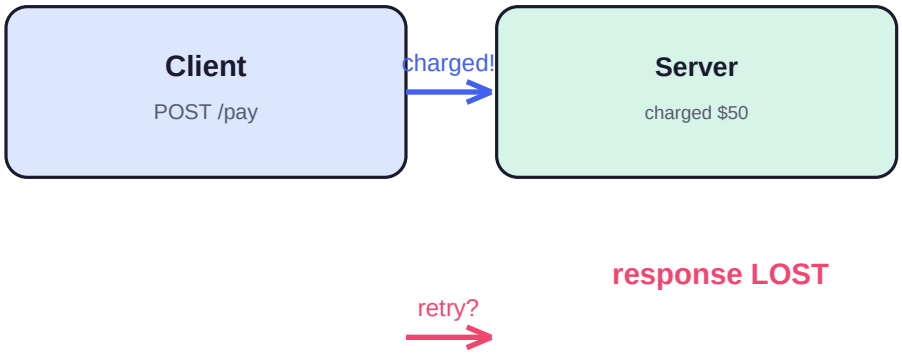
Make retries safe:
keys, dedupe, and exactly-once effects.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Retry Problem

The network failed AFTER the work was done. Was it processed? Unknown.



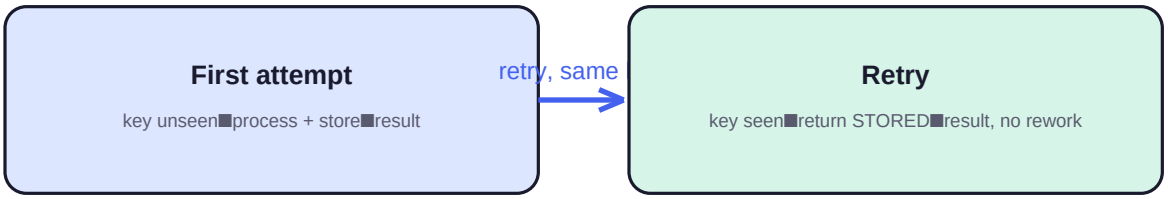
- Timeout does NOT mean failure: the server may have completed the work and died sending the reply.
- Blind retry = double charge. Never retry = lost payment. The escape: idempotency.
- Idempotent: f(f(x)) = f(x) -- doing it twice has the same effect as doing it once.

STEP 2/6

Idempotency Keys

The client generates a unique key per INTENT; the server dedupes on it.

```
POST /pay   Idempotency-Key: 7f3a-...-c91d
```



Stripe's pattern: keys scoped per account, stored 24h+. Same key + different payload = 422 (client bug, surfaced).

STEP 3/6

Safe Retries

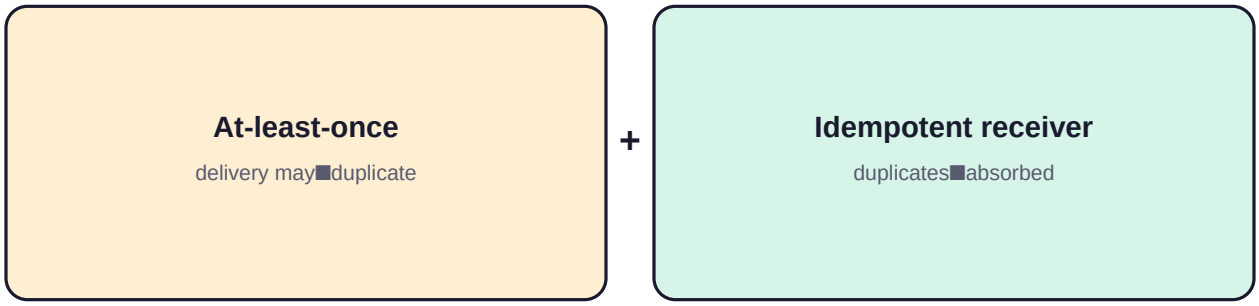
Retry storms are safe when every handler dedupes.

- Client: generate the key ONCE per user intent (button click), reuse it across all retries.
- Server: check-then-act must be ATOMIC -- store the key and the result in one transaction.
- In-flight dedupe: second request with the same key while the first runs should WAIT, not double-process.
- Key TTL: keep keys long enough to cover your max retry window (hours/days, not seconds).

STEP 4/6

Exactly-Once Effects

You cannot have exactly-once DELIVERY -- but you can have exactly-once EFFECT.



= exactly-once EFFECT

This is the industry's practical answer: Kafka transactions + idempotent consumers, SQS + dedupe keys.

STEP 5/6

Naturally Idempotent Design

Some operations are idempotent by construction -- prefer them.

PUT /users/42

set state to X -- repeating sets the same state

DELETE /sessions/9

ensure gone -- already gone is fine

Upserts

INSERT ... ON CONFLICT DO NOTHING -- key-based dedupe in SQL

State machines

transition only from expected state -- replays are no-ops

POST (create) is the dangerous one: 'create order' twice = two orders, unless a key dedupes it.

STEP 6/6

Limits and Sharp Edges

Idempotency is a design contract, not magic.

- Non-idempotent by nature: 'increment counter', 'send email' -- wrap them in dedupe keys or accept duplicates.
- Key scope matters: per-user keys prevent cross-user collisions; per-intent keys prevent double-clicks.
- Clock risk: keys expire -- a retry AFTER expiry replays the work. Size TTLs generously.
- Test it: chaos-test duplicate delivery in staging; every handler should prove it dedupes.

STEP ✓I✓

Key Takeaways

Idempotency — Remember

- Timeouts are ambiguous: the work may have completed -- never blind-retry, never give up.
- Idempotency keys: client-generated per intent, server dedupes atomically and replays stored results.
- Exactly-once EFFECT = at-least-once delivery + idempotent receiver -- the practical standard.
- Prefer naturally idempotent shapes: PUT, DELETE, upserts, state-machine transitions.
- Size key TTLs generously, scope keys per user+intent, and chaos-test duplicate delivery.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev